

13 更好地与设计模式相处

现实世界中的模式



啊哈！现在你已经准备好迎接一个充满设计模式的崭新世界。但是，在你打开所有这些机会的新大门之前，我们需要谈到一些即将在现实世界中遇到的细节。没错，外面的世界比对象村要复杂一些。来吧！从下页开始，我们会指点你从对象村转到现实世界……

对象村指南



更好地与设计模式相处

请接受我们的随身指南。这里提供了一些和现实世界中的模式相处的技巧。在这份指南中，你将会：

- ☞ 了解所有关于“设计模式”定义的常见误解。
- ☞ 发现那些漂亮的设计模式类目，以及了解为什么你只需要一个。
- ☞ 避免在错误的时间使用设计模式的尴尬。
- ☞ 学习如何将模式维持在它所应属的分类中。
- ☞ 认识到发现模式并非只有专家才做得到。阅读我们的速成秘笈，你也能成为一名模式作者。
- ☞ 亲眼目睹神秘的GoF身份被揭露。
- ☞ 能够和邻居有共同的话题，这是一本模式用户在咖啡桌上必聊的书籍。
- ☞ 学习像禅师一样训练你的心智。
- ☞ 通过改进你的模式词汇，结交一些朋友，并影响周围的开发人员。

定义设计模式

我敢说，读完这本书之后，你已经相当了解什么是模式，但我们从未真正给出设计模式的定义。你可能会对于设计模式的常用定义感到有些惊讶：

模式是在某个上下文中针对某个问题的解决方案。

这个定义没有揭示太多东西，对吧？但是别担心，我们现在就一步步地看定义的各部分：上下文、问题和解决方案：

上下文指某个模式适用的情况。这应该是一种会不断出现的情况。

例子：你有一个对象的集合。

问题指在此上下文中你想要达到的目标，但也要考虑该上下文中发生的任何约束。

你需要遍访每个对象，而不暴露集合的实现。

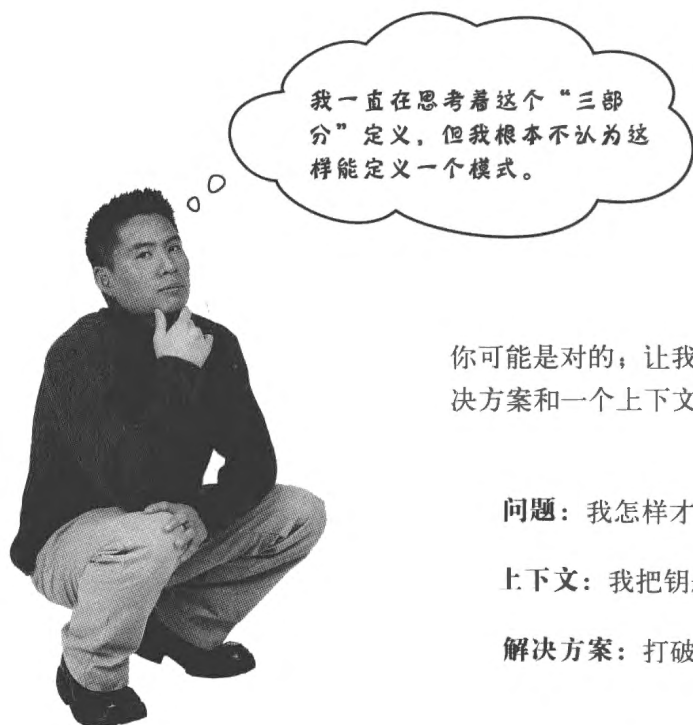
解决方案就是你所追求的东西：一个通用的设计，所有人都可以用来解决目标和约束集。

将迭代封装进分离的类中。

有些定义需要花些时间来理解，这就是其中一个。一步步来，尝试这样想：

“如果你发现自己在某个上下文中，面对一个问题，该问题有一个目标，并受一群约束影响，然后，你能够应用一个设计解决目标和约束，得出一个解决方案。”

现在看来，只是搞清楚什么是设计模式，就已经耗费不少功夫。不过，你已经知道设计模式给了你一个解决方案，来解决常见的、重复发生的设计问题。那为什么还要搞得这么正式？嗯，一会你会看到，用正式的方式描述模式，我们可以创建模式的类目，带来各种好处。



我一直在思考着这个“三部分”定义，但我根本不认为这样能定义一个模式。

你可能是对的；让我们再多想一想……我们需要一个问题、一个解决方案和一个上下文：

问题：我怎样才能准时上班？

上下文：我把钥匙锁在车里了。

解决方案：打破窗户，进入车内，启动引擎，然后开车上班。

定义的组件我们都有了：我们有一个问题，这个问题包括去上班的目标，以及时间、距离的约束，可能还有其他一些影响因素；我们也有一个上下文，车钥匙拿不到；我们有一个解决方案，让我们能够取得钥匙，解决时间和距离的约束。现在，我们肯定有了一个模式！对吧？



我们遵循设计模式的定义，定义了一个问题、一个上下文和一个解决方案（这个方案是行得通的）。这是一个模式吗？如果不是，错在哪儿？我们定义一个OO设计模式的时候，也会犯同样的错误吗？

更近地观察设计模式的定义

我们的例子似乎符合设计模式的定义，但它不是一个真正的模式。为什么呢？对于初学者，我们知道，模式需要应用于重复出现的问题。虽然一个心不在焉的人，可能经常把车钥匙锁在车内，但是一再地打破车窗，这实在称不上是一个可以一次又一次应用的解决方案（或者至少没有平衡另一个约束：成本）。

还有其他一些问题：首先，把这个描述交给别人，让他应用到自己特有的问题上，并不容易。其次，我们违反了模式的一个重要而简单的方面：没有给它起名！没有名字，模式就无法成为开发人员之间共享的词汇。

幸运的是，模式并非只是被描述和归档为简单的问题、上下文和解决方案。我们有更好的方式来描述模式，并将它们收录进模式类目。

下一次，如果有人告诉你，模式是在某个上下文中针对某个问题的解决方案，这个时候你只需点头并微笑。即使对于描述什么是真正的设计模式来说这个定义并不充分，你也知道真正的意思。



没有蠢问题

问：模式的描述是否表达为一个问题、一个上下文，以及一个解决方案？

答：通常你在模式类目中发现的模式描述揭示的内容要比这多一点。我们一会就来详细看看模式类目，模式描述更多是描述某个模式的意图、动机、可能应用该模式的地方、还有解决方案的设计以及使用的后果（好的和坏的）。

问：稍微修改某个模式的结构以符合我的设计，这样可以吗？或者是一定要遵循严格的定义？

答：你当然可以修改。像设计原则一样，模式不是法律或准则，模式只是指导方针，你可以修改模式来符合你的需要。正如你已看到的，现实世界中有许多例子不符合经典的设计模式。

但是，当你在采用模式时，在文档中注明你的模式和经典设计的差异，没有坏处。这样，其他开发人员可以很快地分辨出你使用的模式，以及和经典模式的差异。

问：哪里可以得到模式类目？

答：第一个也是最重要的模式类目是Gamma、Helm、Johnson和Vlissides所著的Design Patterns: Elements of Reusable Object -Oriented Software (Addison-Wesley 出版)。这个类目列出了23个基础模式，后面几页我们会更详细介绍这本书。

各个领域也在开始出版许多其他模式类目，例如：企业软件、并发系统和业务系统。



极客的玩意儿

愿力与你同在

设计模式的定义
告诉我们，问题
包含一个目标和一
组约束。模式大师们
对此有个术语，将其称
为“力”。为什么？这个
嘛，我们确信他们有自己的
理由，但是如果你还记得电
影^{译注1}里说的，力“塑造并控制
宇宙”。类似地，模式定义中的
力也塑造并控制解决方案。只有当
解决方案在力的两边中取得平衡时
（光明面是你的目标，黑暗面是约
束），才算是有用的模式。当你第一次在
模式讨论中看到术语“力”时，可能会感
到十分困惑，但只要记得，力有两面（目标
和约束），它们需要被平衡，以创建模式的解
决方案。别让这个行话挡你的路，愿力与你同在！

译注1：《星球大战》。



Frank: Jim，也算我们一份吧，我们正在到处看文章学习模式。

Jim: 没问题，每个模式类目都包含了一组模式，并详细描述模式与其他模式之间的关系。

Joe: 你是说模式类目不只一份？

Jim: 当然，有基础设计模式类目，也有领域特定模式的类目，例如企业或分布式计算模式。

Frank: 你在看的是哪一份类目？

Jim: 这是经典的GoF类目，包含了23个基础设计模式。

Frank: GoF？

Jim: 没错，意思是四人组。这四个人合作写了第一本模式类目。

Joe: 类目里有什么？

Jim: 有一组相关的模式。每个模式有一份遵循模板的描述，阐述该模式的许多细节。例如，每个模式都有一个名称。

Frank: 哇哦，真了不起，有一个名称！

Jim: 别着急嘛，Frank，事实上，名称真的很重要。当每个模式都有

一个名称时，我们谈论模式就变得容易了，因为有大家共享的词汇。

Frank: 好的，好的。我只是在开玩笑。继续说吧，还有什么？

Jim: 就像我所说的，每个模式都要遵循一个模板。每一个模式都有名称，还有几个段落告诉我们更多关于该模式的知识。例如，有一个段落叫意图（Intent），描述该模式是什么，有点像是定义。然后还有段落叫动机（Motivation）和适用性（Applicability），描述什么时候、什么地方使用这个模式。

Joe: 那么关于设计本身呢？

Jim: 有几个段落是描述类设计，所有组成模式的类以及每个类扮演的角色。也有一个段落描述如何实现该模式，经常有范例代码向你展示怎么做。

Frank: 听起来好像考虑周全。

Jim: 还不只这样。还有模式用在真实系统中的例子。除此之外，我认为最有用的段落之一是：该模式和其他模式之间如何相关。

Frank: 噢，你的意思是说，它们告诉你类似“状态和策略区别在哪”这样的东西？

Jim: 没错！

Joe: 那么Jim，你到底要如何使用这个类目？当遇到问题时，你会查找这份类目来获得解决方案吗？

Jim: 首先，我会尝试熟悉所有模式以及它们之间的关系。然后，当我需要一个模式时，就会想到是哪个模式。我会看看动机和适用性段落，确认我的想法没错。还有一个很重要的段落：结果（Consequence）。我回顾模式的结果，确保模式不会给我的设计带来一些意外的影响。

Frank: 听起来很有道理。因此，一旦你知道这个模式是正确的，你怎么把它应用到你的设计中并实现它？

Jim: 这就是为什么需要类图。我先是阅读结构段落来回顾类图，然后看参与者（Participant）段落，确定我理解每个类的角色。接下来，就可以用到我的设计中，可以对模式做任何想要的修改以符合我的设计。然后，我回顾实现（Implementation）和案例代码（Sample Code）段落，以确认我知道所有良好的实现技巧以及可能碰到的陷阱。

Joe: 我了解类目如何真正让我更快地用上模式了。

Frank: 总结得好。Jim，你能带我们浏览一遍模式描述吗？

类目中所有的模式都以一个名称开始，名称是模式的重要部分——没有好的名称，模式无法成为你和其他开发人员之间共享词汇的一部分。

动机给出一个具体场景，该场景描述问题以及方案如何解决问题。

适用性描述模式可以应用在什么场合。

参与者是设计中的类和对象。这个段落描述它们在模式中的责任和角色。

结果描述用了这个模式之后可能有的效果，包括好的和不好的。

实现提供了实现这个模式时所需要用的技巧，以及你应该留心的问题。

“已知应用”描述已经在真实系统中发现的例子。

SINGLETON

Object Creational

Intent

Es aliquat, velesio ent lore feus acillao rperci tat, quat nonsequam il ea at nim nos do enim qui eratio ex ea faci ter, sequis diom utat, volere magnis.

Motivation

Es aliquat, velesio ent lore feus acillao rperci tat, quat nonsequam il ea at nim nos do enim qui eratio ex ea faci ter, sequis diom utat, volere magnis. Rud modolore dit laoreet augiam iri et dipis diossequis dignibb eumy aibb esequat. Duis nulpotem ipisim esecte consullat wissi.

Os nissisem et lumandre do con el utpatuso coerecipis augue doloreet luptat amet vel iuscident digna feugue dunt num etumy nim dui blaor sequat num vel etue magna augiat. Aliquis nosse vel exer se minissequis do doloretis ad magnit, sim xzrillat ipsummo doctorem dignibb euguer sequam ea am quate magnim illan xzrit ad magna fev facinri deliti ut

Applicability

Duis nulpotem ipisim esecte consullat wissi. Ectem ad magna aliqui blamet, conuallandore dolore magna feus nos alit ad magnim quate modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

Structure

Singleton
static uniqueInstance
if Other useful Singleton data.
static getInstance()
if Other useful Singleton methods.

Participants

Duis nulpotem ipisim esecte consullat wissi. Ectem ad magna aliqui blamet, conuallandore dolore magna feus nos alit ad magnim quate modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

- A dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.
- A feus nos alit ad magnim quate modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

Collaborations

- Feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

Consequences

- Duis nulpotem ipisim esecte consullat wissi. Ectem ad magna aliqui blamet, conuallandore dolore magna feus nos alit ad magnim quate modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.
- Dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.
 - Modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.
 - Dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.
 - Modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

Implementation/Sample Code

Duis nulpotem ipisim esecte consullat wissi. Ectem ad magna aliqui blamet, conuallandore dolore magna feus nos alit ad magnim quate modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

```
public class Singleton {
    private static Singleton uniqueInstance;

    // other useful instance variables here

    private Singleton() {}

    public static synchronized Singleton
    getInstance() {
        if (uniqueInstance == null) {
            uniqueInstance = new Singleton();
        }
        return uniqueInstance;
    }

    // other useful methods here
}
```

Nos alit ad magnim quate modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

Known Uses

Duis nulpotem ipisim esecte consullat wissi. Ectem ad magna aliqui blamet, conuallandore dolore magna feus nos alit ad magnim quate modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

Duis nulpotem ipisim esecte consullat wissi. Ectem ad magna aliqui blamet, conuallandore dolore magna feus nos alit ad magnim quate modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

Related Patterns

Elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er, alit ad magnim quate modolore vent lut luptat prat. Dui blaore min ea feupit ing ent laore magnibb eniat wissasecte et, suscilla ad minicini blam dolorpce rcillit iri, comae dolore dolore et, verci enis enit ip elessequit ut ad esectem ing ea con eros autem diam nonnullu tpatiss isomodignibb er.

这是模式的分类或类目，再过几页就会谈到。

意图简短地描述该模式做什么。你也可以把它想成是模式的定义（就像我们在本书中所用的定义一样）。

结构提供一个图，展示参与模式的类之间的关系。

协作告诉我们，参与者在模式中如何合作。

范例代码提供了代码的片段，可能对你的实现有帮助。

相关模式描述此模式和其他模式之间的关系。

没有蠢问题

问：有可能创建自己的设计模式吗？还是必须成为“模式大师”才能创建？

答：首先，要记住，模式是被“发现”的，不是被创建的。因此，任何人都可以发现某个设计模式，然后写出它的描述，但这不容易，无法速成也不常发生。想成为一个“模式作者”，需要付出努力。

你应该首先考虑一下，为什么你要发现自己的模式——大多数的人都没有编写模式，只是使用模式。不过，你可能工作在某个特定的领域，你认为新的模式会有帮助，或者是你找到一个解决方案，能够解决一个重复出现的问题，或者，你只想要加入模式社群，贡献自己的力量。

问：我有意愿，我要如何开始？

答：和其他科目一样，你知道得越多越好。研究已有的模式，关键是了解它们做什么，以及它们如何和其他模式相关。这不只可以让你熟悉模式是如何打造出来，也可以避免重复制造轮子。接下来，你可以开始在纸上编写你的模式，以便和其他开发人员沟通；关于“如何沟通你的模式”，我们稍后还会谈到更多内容。如果你真的非常感兴趣，可以阅读本Q&A以后的章节。

问：我怎么知道我真的有了一个模式？

答：这个问题很好，除非其他人使用它并且发现它很有用，否则你不能算有了一个模式。一般来说，要通过“三次规则”，才算有了一个模式。这条规则指只有在现实世界的解决方案中应用三次以上，才能叫做一个模式。

你想要当一个设计模式巨星吗？

那么，听清楚了。

找一份模式的类目，然后花些时间好好地学习它。

当你写下一个正确的叙述，而且有三个开发人员都同意你的看法时，然后你就知道它的确是一个模式。



可以用“*So you wanna be a Rock'n'Roll Star*”译注²的调子唱出来。

译注2：美国摇滚乐队The Byrds，飞鸟乐队的歌曲。

想当一个设计模式作者吗？

做好功课。

在你创建新模式之前，你需要精通已有的模式。许多模式看起来是新的，事实上只是已有模式的变体。通过研究模式，你会更善于识别模式，并且知道某个模式和其他模式之间的关系。

花时间反省与评估。

你的经验（你遭遇的问题以及所用的解决方案）是模式想法的来源。因此，花一些时间反省过去的经验，梳理它们，以便使用在新设计上。记住，大多数的模式都是已有模式的变体，而不是新模式。而且当你确实找到了好像是新模式的东西时，它的适用性可能太窄，而不能称得上是一个真正的模式。

用别人能理解的方式，把你的想法写在纸上。

如果其他人不能使用你找到的模式，那么发现新模式的作用就不大；你需要把你的候选模式写成文档，以便其他人能够阅读、理解并应用到他们自己的解决方案中，然后给你反馈。幸运的是，你不需要发明你自己的模式归档方法，你已经见过GoF模板，很多关于如何描述模式及其特征的思想已经在里面了。

让其他人尝试你的模式，然后精炼再精炼。

不要期望一次就把你的模式搞定。把你的模式想像成随着时间推移不断提升的一项工作。让其他开发人员评审你的候选模式，尝试着使用，给你反馈。将反馈汇总到你的描述中，再重复上述的步骤。你的描述永远不会完美，但是到了某个时间点之后，它应该会足够稳定，足以让其他开发人员读懂它。

不要忘了三次规则。

记住，除非你的模式已经成功应用在现实世界的三个解决方案中，否则就不够资格被当成模式。这是另一个很好的理由，让你把你的模式交给别人尝试，并把反馈汇集到模式中。

使用已有的模式模板之一来定义你的模式。这些模板已经包含了许多思想，而且其他模式用户也认识该格式。



★ WHO DOES WHAT? ★

请将下列模式和描述匹配：

模式

描述

装饰者

包装对象，并提供不同的接口。

状态

子类决定如何实现算法中的步骤。

迭代器

子类决定要创建的具体类是哪一个。

外观

确保有且只有一个对象被创建。

策略

封装可以互换的行为，并使用委托来决定要使用哪一个。

代理

客户用一致的方式处理对象集合以及单个对象。

工厂方法

封装了基于状态的行为，并使用委托在行为之间切换。

适配器

提供一种在对象集合中行走而不暴露集合实现的方式。

观察者

简化一群类的接口。

模板方法

包装一个对象，以提供新的行为。

组合

允许客户创建对象的家族，而无需指定它们的具体类。

单件

允许对象在状态变化时被通知。

抽象工厂

包装对象，以控制对此对象的访问。

命令

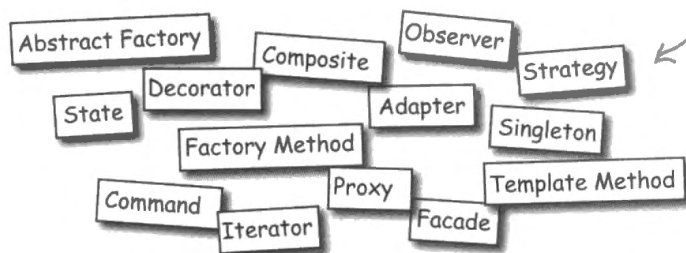
把请求封装为对象。

组织设计模式

随着所发现设计模式的数目逐渐增加，有必要将它们分级分类，这样我们可以把它们组织起来，在搜索设计模式时，不用搜索全部，只需搜索比较窄的范围。另外，同一组内的模式还可以互相比较。

你会发现，在大多数的类目中，模式的分组就是那么几种方式。最广为人知的方式就是第一本模式类目中所使用的方式，根据模式的目标分成三个不同的类目：创建型、行为型和结构型。

削尖你的铅笔

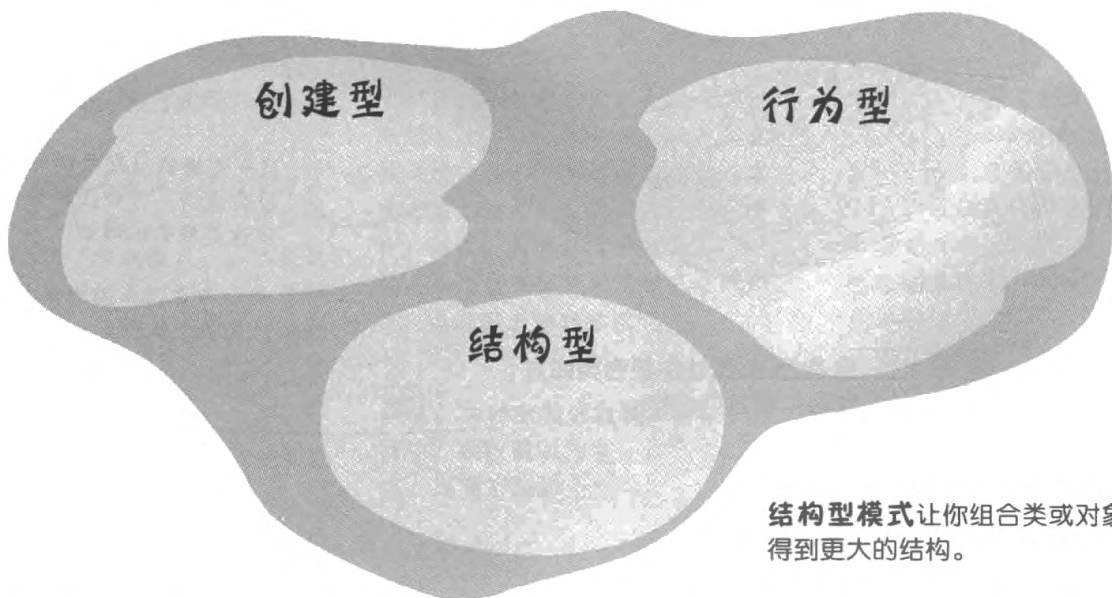


阅读每个模式类目的描述，试着将这些模式正确地归类。这很不容易！但是请尽力而为。正确答案在下一页。

这里的每个模式都属于以下类别之一。

创建型模式牵涉到对象实例化，这类模式都提供一种将客户从需要实例化的对象中解耦的方式。

行为型模式都涉及类和对象如何交互，以及分配责任。



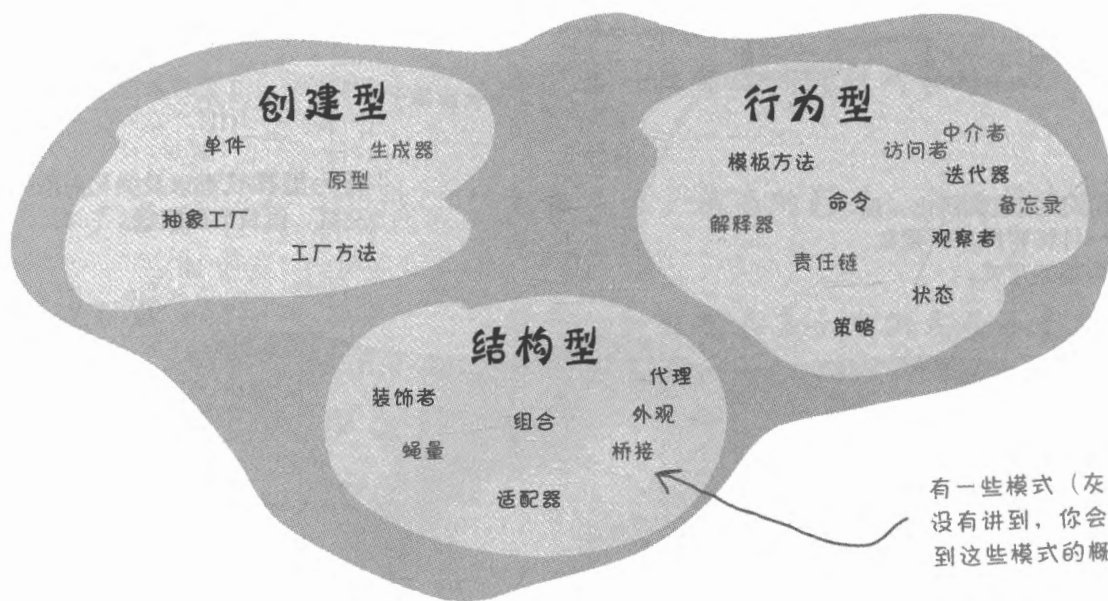
结构型模式让你组合类或对象得到更大的结构。

解答：模式类目

这是把模式分组到类目的结果，你可能觉得这个习题很难，因为许多模式似乎不只符合一个类目，别担心，其实每个人都有这样的困扰。

创建型模式牵涉到对象实例化，这类模式都提供一种将客户从需要实例化的对象中解耦的方式。

行为型模式都牵涉到类和对象如何交互，以及分配责任。

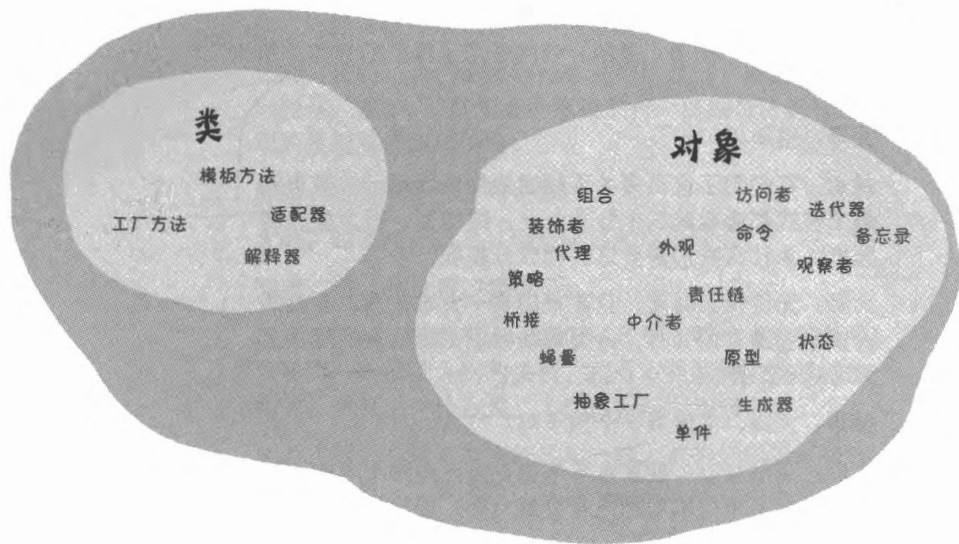


结构型模式让你组合类或对象得到更大的结构。

第二种常用的分类方式是：模式所处理的是类还是对象。

类模式描述类之间的关系如何通过继承定义。类模式中的关系是在编译时建立的。

对象模式描述对象之间的关系，而且对象模式主要通过组合定义。对象模式中的关系通常在运行时创建，更加动态和有弹性。



注意，对象模式比类模式的数目多出许多！

没有蠢问题

问：只有这些分类方式吗？

答：不，还有人提出其他分类方式。有些分类方式先分成三大类，然后再添加小类，像“解耦模式”。你要熟悉最常见的组织模式的方式。如果创建自己的分类方式有助于你更好地理解模式，就放手去做。

问：把模式组织到类目，真的有助于记住它们吗？

答：因为有比较，所以肯定能让你从宏观上把握。但是许多人被创建型、结构型以及行为型类目搞得一头雾水，常有某个模式看起来不只适合一个类目。记住，怎么分类不重要。最重要的是知道这些模式以及模式之间的关系。类目有帮助时才用它！

问：为什么装饰者模式在结构型类目中？我以前以为它应该是行为型模式，毕竟它添加了行为！

答：是的，很多开发人员都这么说！GoF分类背后的想法是这样的：结构型模式用来描述类和对象如何被组合，以创建新的结构或新的功能。装饰者模式能让你通过“用另一个对象来包装对象”来组合对象，以提供新的功能。因此，焦点是如何动态组合对象以获得功能，而不是行为型模式的目的，对象之间的沟通与互连。记住，这些模式的意图是不同的，这常常是理解模式属于哪个类目的关键。



大师和徒弟……

大师：徒弟，你看起来很苦恼。

徒弟：是的，我刚学习了模式的分类，感到很困惑。

大师：继续说……

徒弟：在学习了这么多关于模式的知识之后，我被告知每个模式都适合这三个分类之一：结构型、行为型和创建型。为什么我们需要这些模式分类呢？

大师：无论什么时候，只要我们有一大堆东西，会自然而然想要给它们分类。分类帮助我们在更抽象的层次上思考这些东西。

徒弟：大师，你能举一个例子吗？

大师：当然可以。就拿汽车来说，有许多种不同的汽车款式，我们很自然地把汽车分成几类，像经济型车、跑车、SUV、卡车以及豪华轿车。

大师：你看起来很震惊，难道你没有理解我说的话？

徒弟：大师，我很能理解，我只是对你这么了解汽车感到震惊！

大师：我不能把什么都和莲花或饭钵扯到一起。现在，我可以继续说了吗？

徒弟：是的，是的，很抱歉，请继续。

大师：一旦你有了分类或类目，你就可以很容易地谈论：“如果你想从硅谷走山路开车到圣克鲁斯，那么跑车是最佳选择。”或者“因为石油的市场状况日益恶化，你真的需要买一辆经济型车，比较省油。”

徒弟：因此通过分类，我们可以将一组模式作为一个群体来谈论。如果我们可能知道需要一个创建型模式，但又不知道确切是哪一个，那么依然可以用创建型模式来统称。

大师：是的，而且分类提供了一种将一个成员与该类目其他成员比较的方式。例如，“Mini是最时髦的小型车。”或者帮助我们缩小搜索范围，“我需要一部省油的车。”

徒弟：我了解了。因此我可以说，“对于改变对象的接口，适配器模式是最好的结构型模式”。

大师：是的，我们也可以把类目用作另一个目的：开发新领域，例如，“我们真的想要交付一部跑车，具有法拉利的性能和本田的价格。”

徒弟：听起来像是个死亡陷阱。

大师：对不起，徒弟，我没听清楚你说什么。

徒弟：唔，我是说“我懂了。”

徒弟：因此，类目有助于我们思考模式群组之间的关系，以及同一组模式之间的关系，还可以帮助我们推导新的模式。但是，为什么是三个类目，而不是四个或五个？

大师：啊，就像是夜空的星星一样，如果你需要，可以有很多类目。三这个数比较省事，而且是许多人所决定出来给模式分组的好数目。但是也有其他人建议用四个、五个或者更多。

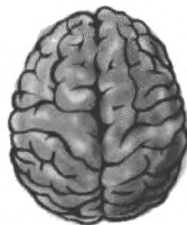


用模式思考

上下文、约束、力、类目、分类……好家伙，听起来挺学术的。

OK，这一切都很重要，知识就是力量。但是，我们必须面对一个事实，如果你了解学校学的东西，而没有使用模式的经验和实践，那么这不会给你的人生带来多大的差别。

下面是一份快速指南，可以帮助你开始“用模式思考”。什么意思？就是能够看着设计，并体会什么地方模式能自然适用，哪些则不能。



你思考模式的大脑

保持简单（Keep It Simple & Stupid, KISS）

首先，当你设计时，尽可能用最简单的方式解决问题。你的目标应该是简单，而不是“我如何在这个问题中应用模式”。千万不要有这样的感觉，如果没有用模式来解决问题，你就不是经验丰富的开发人员。如果你能把设计做得简单，其他开发人员会欣赏你的。应该这样说，为了保持你的设计简单和有弹性，有时候使用模式是最好的方式。

设计模式不是万灵丹；事实上，什么丹都不是！

正如你知道的，模式是解决重复发生的问题的通用解决方案。模式还有一个好处，它被大量开发人员测试过。因此，当你需要某个模式时，可以高枕无忧地使用它，毕竟之前很多开发人员已经使用类似技巧来解决问题。

不过，模式不是万灵丹。你不能把模式插入，编译，然后就早早地去吃午餐。要使用模式，你还需要全面考虑模式对你的设计中其他部分所造成的后果。

你知道什么时候需要模式……

啊……这是最重要的问题：什么时候使用模式？当你在设计时，如果确定某个模式能解决你的设计中的问题，那就引进这个模式。如果有更简单的解决方案，那么在决定使用模式之前应该先考虑这个方案。

知道什么时候适用一个模式，这就是你的经验和知识起作用的地方。一旦你确定一个简单的解决方案无法满足你的需要，你应该考虑这个问题以及相关的一组约束，这会帮你将问题匹配到模式。如果你对模式很了解，就可能知道有什么模式适合这样的情况。否则，调查一下看起来有可能解决问题的模式。模式类目中的意图和适用性段落会特别有用。一旦找到了一个看起来匹配的模式，先确定这个模式所带来的后果，以及对设计其他部分的影响是你所能承受的。如果一切看起来都很好，就用它吧！

有一种情况，即使更简单的解决方案可行，你还是想要使用模式，当你预测系统某方面会变化时。正如我们已经看到的，认识到你的设计中有变化的区域，通常是需要模式的迹象。只是要确定，你添加模式是为了应对实际可能要发生的变化，而不是假想的变化。

并非只有在设计时才考虑引进模式，重构时也要这样做。

重构时间就是模式时间！

重构就是改变你的代码和改进它的组织方式的过程。目标是改善其结构，而不是其行为。这是一个很好的时机，可以重新检查你的设计，看看是否能够用模式让它拥有更好的结构。例如，充满条件语句的代码，可能意味着需要使用状态模式，或者，应该用工厂模式清除具体的依赖。

有很多书的主题是关于如何用模式来重构的，随着你的经验增长，你要进一步学习这方面的知识。

拿掉不是真正需要的。不要害怕从你的设计中移除设计模式。

还没有人谈到什么时候要移除某个模式。移除模式，这是亵渎神灵啊！不，我们都是成人了，应该要面对这个问题。

因此，什么时候你应该移除某个模式？当你的系统变得非常复杂，而且计划中不需要任何弹性的时候。换句话说，当一个更简单、没有模式的解决方案会更好的时候。

如果你现在不需要，就别做。

设计模式威力强大，你很容易看到，各种模式可以用到你当前的设计中。开发人员天生就热爱创建漂亮的架构，准备好应对来自任何方向的改变。

要抗拒这样的诱惑。如果你今天在设计中有实际的需要去支持变化，就放手采用模式处理这个变化。但是，如果理由只是假想的，就不要添加这个模式，因为它只会增加你系统的复杂度，可能你永远都不会需要它！

将你的思考集中于设计而不是模式。有自然的需要时才使用模式。如果更简单的东西行得通，就用更简单的。





大师和徒弟……

大师：徒弟，你的初步训练差不多完成了，接下来的计划是什么？

徒弟：我要去迪斯尼！然后开始用模式创建许多代码！

大师：哇哦，等等。杀鸡焉用牛刀啊！

徒弟：这是什么意思呢，大师？我已经学会了设计模式，难道不应该在我的设计中都用上以达到最强的威力、弹性和可维护性吗？

大师：不，模式是一个工具，一个只有在需要时才用的工具。你也花了很多时间学习设计原则。总是从你的原则开始，创建最简单的代码以完成工作。不过，如果你看到有需要模式的地方，就使用模式。

徒弟：所以我不应该从模式开始建造我的设计？

大师：这不应该是开始设计时我们的目标，要让模式在你的设计过程中，自然而然地出现。

徒弟：既然模式这么棒，为什么使用时还得如此小心？

大师：模式会引进复杂性，如果没有必要，我们绝不需要复杂性。但是，在需要时，模式是威力强大的。正如你已经知道的，模式是一种被证实过的设计经验，可以用来避免常见的错误。模式也是一种共享的词汇，让我们和其他人沟通我们的设计。

徒弟：那么，我们怎么知道什么时候应该引进设计模式呢？

大师：当你确信你的设计中有一个问题需要解决，或者当你非常确信将来需要处理应用的需求变化时，就引进模式。

徒弟：我觉得我的学习应该继续，即使我已经理解了许多模式。

大师：是的。学会管理软件的复杂度和变更，这是一生的追求。但是现在既然你已经知道许多模式，就可以开始在你的设计需要的地方应用，并持续学习更多的模式。

徒弟：等一下，你的意思是，我还没有了解所有模式？

大师：徒弟，你已经学会一些基本的模式，你会发现还有更多的模式，包括一些应用在特定领域的模式，例如并发系统和企业系统。不过，现在你已经掌握了基础知识，学会这些模式不会太难。

模式的心智



初学者的心智

“我要为Hello World找个模式”

初学者到处使用模式。这是好事，初学者可以获得很多使用模式的实际经验。初学者也认为，“我用的模式越多，我的设计就越好”。初学者会慢慢认识到并非如此，所有设计都应该尽可能简单。只有在实际需要扩展性的地方，才用到复杂和模式。

随着学习的进程, 中阶者的心智开始能看到什么地方需要模式, 什么地方不需要。中阶者的心智依然会试图把过多模式套用在不适当的地方, 但他们也开始察觉到, 典型模式不适用时, 可以调整模式以适合目前的情况。



中阶者的心智

“也许这里我需要一个单件”



悟道者的心智

“这里用装饰者很自然”

悟道者的心智能够看到模式在何处能够自然融入。悟道者的心智不痴迷于使用模式，而是寻找最能解决问题的简单解决方案。悟道者的心智会思考对象的原则及其折中。当模式的需要自然出现时，悟道者的心智会很好地把模式适配到当前的情况。悟道者的心智也看到相似的模式之间的关系，以及相关模式在意图上的微妙差异。悟道者的心智也是初学者的心智，它不会让模式的知识过度影响设计决策。

警告：过度使用设计模式可能导致代码过度工程化。应该总是用最简单的解决方案完成工作，在需要的地方才引进模式。

等一下，我已经读了整本书，你现在却告诉我不要使用模式？



当然我们希望使用设计模式！

但是我们更希望你能够成为一个好的OO设计者。

当一个设计决策要使用某个模式时，你就受益了，因为你使用的解决方案是被许多开发人员验证过的，而且这个方案被很好地归档，其他开发人员会容易认识（你知道的，所有人共享的词汇）。

但是，使用设计模式也有缺点。设计模式引进额外添加的类和对象，因此会增加设计的复杂度。设计模式也会给你的设计添加更多的层，这不但增加复杂度，而且效率下降。

还有，有时候使用设计模式是小题大做。很多时候，你可以回头看看设计原则，会发现有很多简单的解决方案能解决相同的问题。如果是这样，不要抗拒。就用更简单的解决方案。

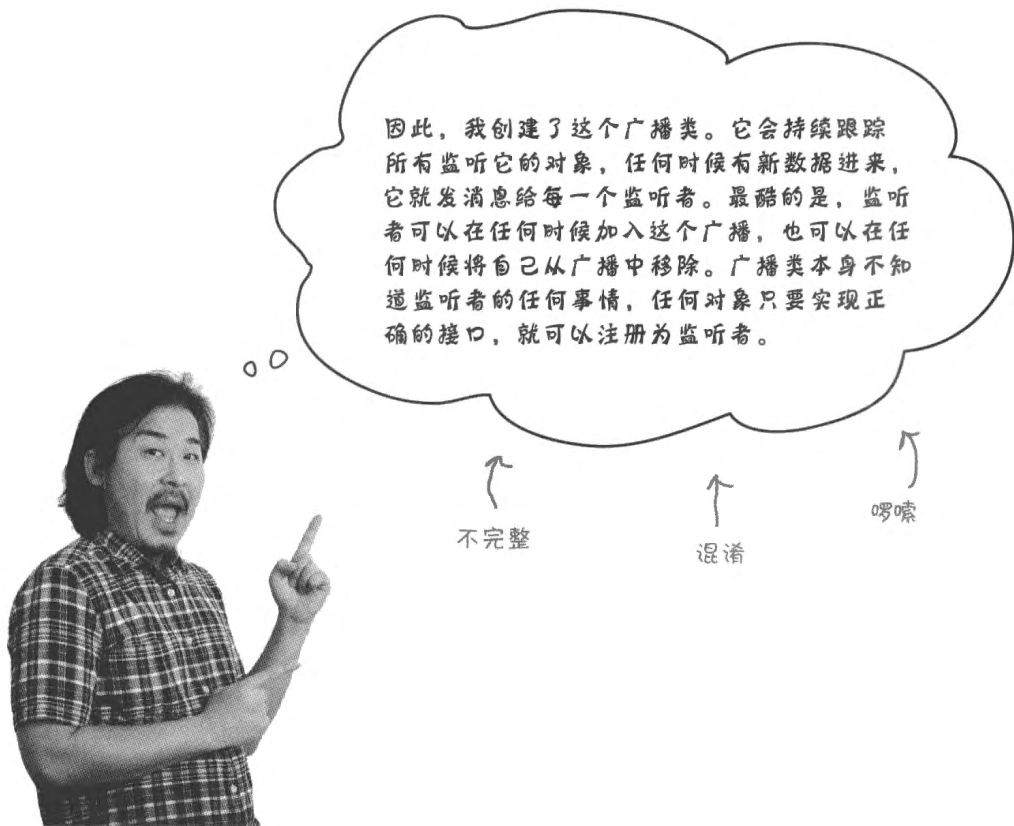
不过，不要因我们的话感到挫折。当设计模式应用得当时，好处很多。

别忘了共享词汇的威力

在这本书中，我们花了如此多的时间讨论OO的基础知识，别忘了设计模式中人的一面，设计模式不只有助于往你在脑袋里塞解决方案，也可以让你和其他开发人员之间有共享的词汇，这是设计模式最大的优点之一。

想想看，从上次我们谈到共享词汇到现在，有些东西已经变了；你现在已经开始创建自己的某些词汇！更别说你还学会了一整套OO设计原则，而从这些设计原则，你可以轻易理解所遇到的任何新模式的动机和运作方式。

既然你已经有了设计模式的基础，是时候到外面去向其他人散播这些词汇了。为什么呢？因为当你的开发人员小伙伴也知道模式并使用共享词汇，你们的设计会更好，沟通更容易。最棒的是，为你省下大量时间去做更酷的事情。



共享词汇的五种方式

- 1. 在设计会议中:** 当你和你的团队在会议中讨论软件设计时, 使用设计模式有助于停留“在设计中”久一点。从设计模式以及OO原则的视角讨论设计, 可以避免你的团队陷入实现的细节, 也可以避免许多误解。
- 2. 和其他开发人员:** 在和其他开发人员的讨论中使用模式。这可以帮助其他开发人员学习新模式, 并建立一个社群。分享你所学会的东西的最美妙之处在于其他人“有收获了”的感觉。
- 3. 在架构文档中:** 在写架构文档时, 使用模式会减少文档的篇幅, 让读者更清楚地了解你的设计。
- 4. 在代码注释以及命名习惯上:** 在编码时, 在注释中清楚地注明你所使用的模式, 还有在选择类和方法的名称时, 尽可能揭露出隐藏在下面的模式。其他开发人员在不得不阅读你的代码时, 会感激你让他们快速理解你的实现。
- 5. 将志同道合的开发人员集合在一起:** 分享你的知识。许多开发人员听说过模式, 但没有很好理解什么是模式。搞一场模式自带午餐会或者在本地用户组演讲。



和GoF一起巡游对象村

在对象村内，你不会碰到“杰茨帮”和“鲨鱼帮”^{译注3}，但是你可能会遇到四人组。你大概也注意到了，想要在模式的世界中走得够远，不可避免要遇到他们。那么，这个神秘的帮派是怎么一回事？

简单地说，GoF包括了Erich Gamma、Richard Helm、Ralph Johnson和John Vlissides。他们归纳了第一本模式类目，而这个过程开启了波及整个软件领域的运动。

这个称号又是怎么来的？没有人知道，反正大家一直就这么叫。但是想想看：如果你想要对象村笼罩上一些“帮派元素”，还有比他们更好的帮派吗？事实上，他们甚至同意来拜访我们……

GoF发起了软件模式运动，但其他许多人也做出了重大的贡献，包括Ward Cunningham、Kent Beck、Jim Coplien、Grady Booch、Bruce Anderson、Richard Gabriel、Doug Lea、Peter Coad和Doug Schmidt，上面说的只是其中一些。



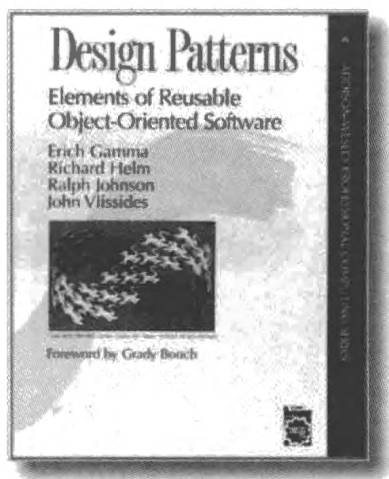
译注3：电影《西区故事》中的两个帮派。



John Vlissides于2005年辞世，这是设计模式社群的巨大损失。

你的旅途刚开始……

现在你站在设计模式的顶部，准备挖得更深，有三本权威书籍，你需要添加到你的书架上……



设计模式的权威书籍

这本书在1995年出版，是整个设计模式领域的第一本。你可以在这本书中找到所有的基础模式。事实上，《Head First 设计模式》所用的模式集合就是基于这本书。

这本书不是设计模式的终点——这本书出版以来，这个领域扩大了很多，不过，这本书是第一本也是最权威的一本。

在你读完《Head First 设计模式》之后，拿起《设计模式》继续探索是一个很棒的选择。

这本书的作者被大家亲切地叫作
四人组，或简称GoF。



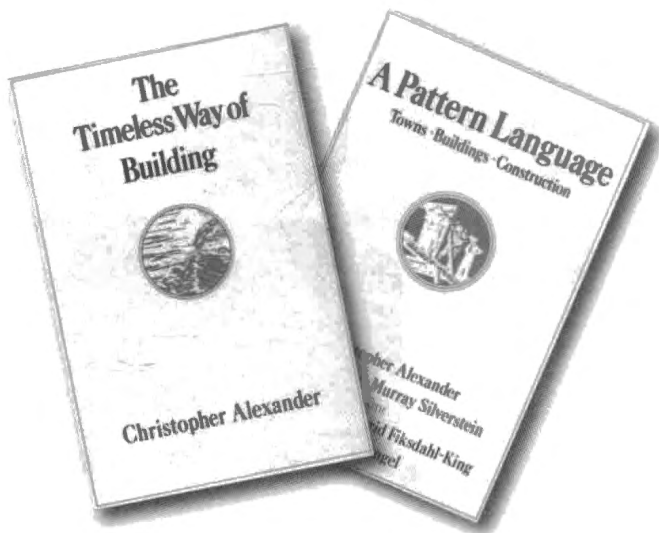
Christopher Alexander发明了模式，
启发其他人把类似的解决方案应用
到软件上。



模式的权威书籍

模式并非从GoF开始，而是从Christopher Alexander开始。他是一位伯克利的建筑学教授，没错，Alexander是个建筑师，不是计算机科学家。Alexander发明了建造有活力建筑的模式（像房屋、城镇和城市）。

下次当你有心情挖掘得更深入时，可以拿起《The Timeless Way of Building》和《A Pattern Language》来阅读。你会看到设计模式的真正起源，并认识到创建“有活力的”建筑和有弹性、可扩展性的软件之间的相似之处。因此，端起一杯星巴兹咖啡，坐下来享受这一切吧……



其他设计模式资源

你会发现外面有许多的生机勃勃而友善的模式用户和作者社群，他们在欢迎你的加入。你可以从以下这些资源开始……



网站

The Portland Patterns Repository，由Ward Cunningham运作，这是一个致力于所有模式相关内容的Wiki。你会发现，每一个你能想到的和模式及OO系统相关的主题在这里都有讨论。

<http://c2.com/cgi/wiki?WelcomeVisitors>

Hillside Group，致力于促进通用的编程和设计实践，并为模式工作提供作集中的资源。这个网站包含了许多模式相关资源的信息，例如文章、书籍、邮件列表和工具。

Hillside.net

O'Reilly Online Learning 提供在线的设计模式书籍、课程和直播教学。你会发现还有基于本书的设计模式训练营课程。

oreilly.com



会议和研讨会

如果你想和模式社群互动，务必去查看众多模式相关的会议和研讨会。Hillside网站持续更新完整的清单。查看一下PLoP（程序的模式语言，Pattern Languages of Programs）以及ACM的OOPSLA（面向对象系统、语言和应用，Object-Oriented Systems, Languages and Applications），现在是SPLASH会议的一部分。

其他资源

如果不提到下面的内容，就是我们的失职了。在Google、Stack Overflow、Quora和很多其他网站和服务都可以询问设计模式问题，找到答案并讨论设计模式。当然，对于你收到的Web上的任何信息，都需要再三检查。

模式动物园

正如我们刚刚所说的，模式并非从软件开始；而是从建筑和城镇的架构开始。事实上，模式的概念可以应用在许多不同的领域。我们来逛逛模式动物园，看看有哪些……



建筑模式 用来创建生气勃勃的建筑、城镇和城市的架构。这也正是模式开始的地方。



栖息地：可以在你所居住、观看、参观的建筑物中发现。



栖息地：出现在三层架构、客户/服务器系统以及Web中。

应用模式是创建系统级架构的模式。许多多层架构都属于这个类目。



野外笔记：MVC被当成应用模式广为人知。



领域特定模式 关注领域特定的问题，像并发系统或实时系统。



帮助找到它的栖息地

企业计算

业务流程模式 描述业务、顾客和数据之间的交互，可以应用于如何高效决策并沟通决策等问题。



出设在公司的会议室和项目管理会议中。

帮助找到栖息地

开发团队
顾客支持团队

组织模式描述了人类组织的结构以及运作。大多数工作聚焦于生产和/或支持软件的组织。



用户界面设计模式 致力于如何设计交互式软件程序的问题。

栖息地：可在视频游戏设计者、GUI构造者和制作者附近发现。

野外笔记：请将你对模式领域的观察添加在这里：

用反模式消灭恶势力

如果我们只有模式，而没有反模式，那么这个宇宙就不完整了。

如果设计模式针对某个特定上下文中重复出现的问题给你一个通用的解决方案，那么反模式能给你什么？

反模式告诉你如何从问题到达一个坏的解决方案。

你可能会问自己，“怎么有人会浪费时间来把坏解决方案归档？”

可以这样想：如果不断有人用坏解决方案来处理某个常见问题，那么，通过把它归档，我们可以防止其他开发人员犯同样的错误。毕竟，避免坏解决方案，与发现好解决方案一样有价值！

我们来看看一个反模式的元素：

反模式告诉你为什么坏解决方案会有吸引力。必须承认，如果坏解决方案看起来没有任何吸引力，不会有人选择它。反模式最重要的工作之一就是警告你提防这种解决方案的诱惑。

反模式告诉你为什么这个解决方案从长远来看是不好的。为了理解为什么这是一个反模式，你必须理解它在未来如何造成负面影响。反模式描述使用这个解决方案时将来什么地方会有麻烦。

反模式向你建议能提供好的解决方案的其他合适模式。要真正带来帮助，反模式需要为你指出正确的方向；它应该建议引向好的解决方案的其他可能性。

我们来看一个反模式。



反模式总是看起来像是一个好解决方案，应用它之后，才会发现是个坏解决方案。

通过将反模式归档，我们帮助其他人在实现坏解决方案之前，就识别出它们。

像模式一样，有很多类型的反模式，包括开发反模式、OO反模式、组织反模式和领域特定反模式。

下面是一个软件开发反模式的例子。



反模式

就像设计模式一样，每个反模式有一个名称，因此我们可以创建共享词汇。

名称：黄金榔头

问题与上下文，和设计模式的描述一样。

问题：你需要为你的开发选择技术，你相信正好有一种技术能够主宰这个架构。

上下文：你需要开发一些新系统或软件，但是所开发的系统或软件却无法和开发团队所熟悉的技术相吻合。

告诉你为什么这个解决方案有吸引力。

力：

- 开发团队倾向于他们懂得的技术。
- 开发团队不熟悉其他技术。
- 采用不熟悉的技术被认为风险比较高。
- 使用熟悉的技术做开发，比较容易规划和预估。

坏的，但有吸引力的解决方案。

原本的解决方案：不管怎样，就用熟悉的技术好了。像着魔似的，应用熟悉的技术来解决很多问题，包括明显不适合的地方也照用。

如何到达一个好的解决方案。

重构的解决方案：通过教育、训练以及读书会，开发人员可以拓展知识，学会新的解决方案。

举例可以观察到反模式的地方。

例子：

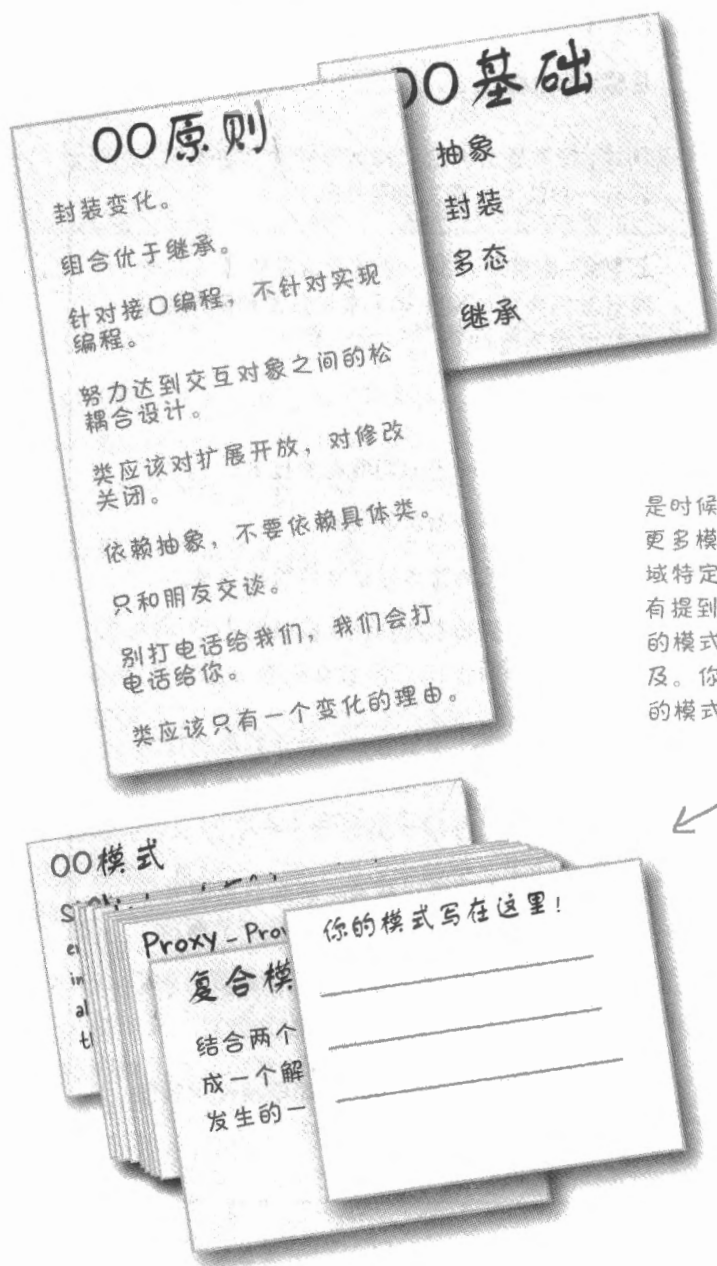
当使用开源替代品时，Web公司坚持使用和维护他们内部自行开发的缓存系统。

修改自Portland Pattern Repository的Wiki，网址在<https://wiki.c2.com/?WelcomeVisitors>，在那里你会发现许多反模式以及相关讨论。



你的设计工具箱中的工具

你已经超过我们的临界点，现在是时候走向外面的世界，自行探索模式了……



是时候自己走出去发现更多模式了。有许多领域特定模式，我们还没有提到，还有一些基础的模式，我们也没有涉及。你也可以创建自己的模式。

去看看附录，有一些更基础的模式，你可能会感兴趣。

要点

- 让设计模式出现在你的设计中，而不是为了使用而使用。
- 设计模式并非僵化的教条；可以采纳或调整以符合你的需要。
- 总是使用满足需要的最简单解决方案，即使方案里没有包含模式。
- 学习设计模式的类目，可以帮你自己熟悉模式以及它们之间的关系。
- 模式的分类（或类目）为模式提供了分组，如果分组对你有帮助，就使用。
- 成为一名模式作者，你需要付出努力：这需要时间和耐心，还必须乐意做大量的精化工作。
- 记住：你所遇到大多数的模式都是已有模式的改编，不是新模式。
- 建立你的团队的共享词汇，这是使用模式最大的好处之一。
- 和其他社群类似，模式社群也有自己的行话。别让这些行话挡住你的路，读完了这本书，现在你已经知道大多数行话了。

离开对象村……



亲爱的，对象村有你们真好。

我们一定会想念你们的。但是，别担心，下一本Head First书很快就会出版，到时候欢迎你们再度来访。你问我下一本书是什么，嗯……真是好问题！要不帮我们参谋一下？发邮件到booksuggestions@wickedlysmart.com吧。

WHO DOES WHAT?

答案

请将下列模式和描述匹配：

模式

描述

装饰者

状态

迭代器

外观

策略

代理

工厂方法

适配器

观察者

模板方法

组合

单件

抽象工厂

命令

包装对象，并提供不同的接口。

子类决定如何实现算法中的步骤。

子类决定要创建的具体类是哪一个。

确保有且只有一个对象被创建。

封装可以互换的行为，并使用委托来决定要使用哪一个。

客户用一致的方式处理对象集合以及单个对象。

封装了基于状态的行为，并使用委托在行为之间切换。

提供一种在对象集合中游走而不暴露集合实现的方式。

简化一群类的接口。

包装一个对象，以提供新的行为。

允许客户创建对象的家族，而无需指定它们的具体类。

允许对象在状态变化时被通知。

包装对象，以控制对此对象的访问。

把请求封装为对象。